REDIS ROADMAP · PHASE 5

# AI / ML Redis

*Caching that makes AI applications fast, cheap, and production-ready*

An end-to-end, self-contained handbook. Read it top to bottom and you will understand every piece of Phase 5 — the four AI/ML caching patterns and the fully-working cached RAG chatbot project that ties them together.

### What's inside


1. Why caching matters for AI (the big picture)
2. Concept 1 — LLM Response Cache
3. Concept 2 — Embedding Cache
4. Concept 3 — RAG Retrieval Cache & Invalidation
5. Concept 4 — Feature Store Cache
6. Project 4 — The Cached RAG Chatbot (architecture)
7. Running everything & what to observe
8. Production checklist & key takeaways


Prerequisite: Phases 1–4 complete · Runs fully offline (no API key required)

CHAPTER 1

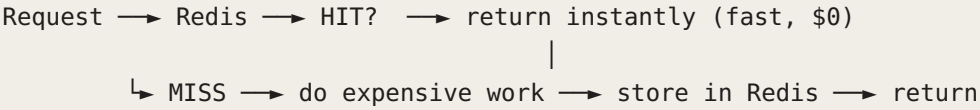
# Why Caching Matters for AI

In Phases 1–4 you cached *database* results. AI workloads add two new pressures that make caching even more valuable: **every model call is slow** (often seconds) and **every model call costs money** (you pay per token). In a real application the same work is requested over and over, so recomputing it every time wastes both.

Phase 5 applies the caching patterns you already know to four AI-specific jobs:

| Cache | What it stores | Why it's safe to cache |
|---|---|---|
| **LLM response** | The model's answer to a prompt | Same prompt → same answer (at temperature 0) |
| **Embedding** | The vector for a piece of text | Deterministic: same text → identical vector, forever |
| **Retrieval** | Documents returned for a query | Stable *until* the knowledge base changes |
| **Feature** | Pre-computed ML features per entity | Computed offline, read at millisecond speed |

> **THE ONE MENTAL MODEL**
>
> Every cache in this phase is the **Cache-Aside** pattern from Phase 1: *look in Redis first; on a miss, do the expensive work, then store the result for next time.* If you understand that one loop, you understand all of Phase 5.




The rest of this handbook walks through each cache with a runnable example, then combines the first three into a single production-style RAG chatbot.

CHAPTER 2 · CONCEPT 1

# LLM Response Cache

file `concepts/01_llm_cache.py`

## The problem

A single LLM call takes 1–10 seconds and costs money per token. Yet users ask the same questions constantly ("What is your refund policy?" might be asked thousands of times a day). Calling the model every time is pure waste when the answer doesn't change.

```
Question → Redis → MISS → LLM → store → answer
Question → Redis → HIT  → answer   (no LLM call at all)
```

## The key idea: hash the prompt into the cache key

Two byte-identical prompts should share one answer, so we turn the prompt into a short fingerprint with SHA-256. We fold in the **model name** and **temperature** too, because the same prompt on a different model — or at a different temperature — is really a different request.

```
def make_llm_cache_key(prompt, model, temperature):
    raw = json.dumps({"prompt": prompt, "model": model,
                      "temperature": temperature}, sort_keys=True)
    digest = hashlib.sha256(raw.encode()).hexdigest()
    return f"llm:response:{digest}"     # short, unique, stable
```

## The cache-aside wrapper

```
def ask_llm(prompt, model="gpt-4", temperature=0.0, ttl_seconds=3600):
    key = make_llm_cache_key(prompt, model, temperature)

    cached = r.get(key)                 # STEP A: read cache
    if cached is not None:
        return cached                   # HIT — instant, free

    answer = call_llm(prompt, model, temperature)  # STEP B: expensive work
    r.set(key, answer, ex=ttl_seconds) # STEP C: store with TTL
    return answer
```

> **TTL = YOUR DEFENSE AGAINST STALE ANSWERS**
>
> `ex=3600` makes Redis auto-delete the answer after one hour, so if the underlying knowledge changes, users eventually get a fresh response. Short TTL = fresher but fewer hits; long TTL = more hits but staler. You tune this per use case.

## What you'll observe when you run it

| Call | Result | Time |
|---|---|---|
| 1st (same question) | **CACHE MISS → LLM** | ~2.0 s |
| 2nd (same question) | **CACHE HIT → Redis** | ~0.0 s |
| 3rd (new question) | **CACHE MISS → LLM** | ~2.0 s |

Same answer, but the repeat is essentially free and instant.

> **PRO TIP**
>
> Only cache *deterministic* calls (temperature 0). If you set a high temperature because you *want* varied output, caching would defeat the purpose — so you might skip the cache for those requests.

CHAPTER 3 · CONCEPT 2

# Embedding Cache

file `concepts/02_embedding_cache.py`

## What an embedding is

An embedding turns text into a list of numbers (a vector) that captures its *meaning*. Similar texts get similar vectors, which is what powers search, RAG, clustering, and recommendations.

```
                "dog"   → [0.12, -0.44, 0.98, ...]
"puppy" → [0.13, -0.41, 0.95, ...]   (very close to "dog")
     "car"   → [0.88,  0.02, -0.31, ...]  (far away)
```

## Why embeddings are the perfect thing to cache

Generating an embedding costs money and time — but the mapping is **100% deterministic**: the same text always produces the exact same vector for a given model. So re-embedding the same text is total waste, *and* there is zero risk of staleness. This is the safest cache in the whole phase.

> **NO TTL NEEDED**
>
> Because `text → vector` never changes for a fixed model, embeddings can live in Redis indefinitely. When memory runs low, Redis's eviction policy (LRU / LFU — remember Phase 1?) removes the least-used ones automatically.

## Batch embedding: only compute the misses

In real pipelines you embed many texts at once. A smart cache asks Redis for all of them in a **single round trip** with `MGET`, then sends only the missing ones to the model.

```
keys = [make_embedding_key(t, model) for t in texts]
cached_values = r.mget(keys)        # ONE network call, not N

missing = []
for text, cached in zip(texts, cached_values):
    if cached is not None:
        results[text] = json.loads(cached)   # HIT
    else:
        missing.append(text)                 # needs computing

for text in missing:                # compute + store only misses
    vec = embed_text(text, model)
    r.set(make_embedding_key(text, model), json.dumps(vec))
```

> **WHY MGET MATTERS AT SCALE**

> Calling `GET` in a loop makes N network round trips. `MGET` does it in one. At thousands of items, that is the difference between milliseconds and seconds.

CHAPTER 4 · CONCEPT 3

# RAG Retrieval Cache & Invalidation

file `concepts/03_rag_cache.py`

## What RAG is

RAG (Retrieval-Augmented Generation) grounds the LLM in *your* data. Instead of hoping the model knows the answer, you retrieve relevant documents from a knowledge base and feed them into the prompt as context.

```
Question → [Retriever] → Vector DB → top-k documents
                └→ documents + question → LLM → grounded answer
```

The retrieval step is a similarity search over possibly millions of vectors — it costs CPU/time and, for hosted vector DBs, money per query. The same question retrieves the same documents, so we cache the result.

## The hard part: invalidation

Unlike embeddings, retrieval results **can go stale**: if you add or edit documents, old cached results become wrong. Phase 5 uses *two* defenses together.

### Defense 1 — TTL (the safety net)

Store retrievals with a short expiry (e.g. 5 minutes). Even if you forget to invalidate manually, stale entries self-destruct.

### Defense 2 — Versioning (the instant purge)

Keep a single "knowledge base version" number in Redis and bake it into every cache key. When the KB changes, `INCR` the version — now the same query produces a *different* key, so every old entry is instantly orphaned. One atomic operation invalidates the entire retrieval cache.

```
def make_retrieval_key(query, top_k):
    version = get_kb_version()               # e.g. 1
    raw = f"v{version}::k{top_k}::{query}"
    return f"rag:retrieval:{sha256(raw)}"

def bump_kb_version():                        # call after re-indexing
    return r.incr("rag:kb_version")           # v1 → v2, all old keys dead
```

| Event | Retrieval cache |
|---|---|
| 1st query | **MISS → Vector DB** |
| 2nd query (same) | **HIT → Redis** |

| | |
|---|---|
| after `bump_kb_version()` | **MISS again (version changed)** |

> **THE RULE TO REMEMBER**
>
> TTL + versioning is the production-grade answer to cache invalidation: versioning gives you an *instant, global* purge; TTL guarantees nothing stays stale forever even if you forget to purge.

CHAPTER 5 · CONCEPT 4

# Feature Store Cache

file `concepts/04_feature_store_cache.py`

## What a feature store is

A "feature" is an input signal an ML model uses to predict — e.g. for fraud detection: transactions in the last hour, average amount, account age. A feature store serves these features to models. It has two sides:

- **Offline** — big historical tables used for *training*.
- **Online** — a millisecond-fast key-value store used at *prediction* time. **This is where Redis shines.**

```
write side (batch/stream job) → Redis → read side (model serving)
    heavy compute done ONCE            fast reads, single-digit ms
```

## How it differs from the other caches

The LLM/embedding/retrieval caches "compute on miss." A feature store is usually **pre-computed and pushed**: a batch or streaming job does the heavy aggregation ahead of time, and the model-serving path only ever *reads*. Redis here is the low-latency serving layer, not a fallback.

## One hash per entity

We store all of an entity's features in a single Redis **hash**, so related data lives together and can be read in one call.

```
# WRITE side (runs offline / on a stream)
r.hset("features:user:123", mapping={
    "avg_txn_amount": "45.20",
    "txns_last_hour": "12",
    "new_country":    "true",
})
r.expire("features:user:123", 3600)   # optional TTL for time-bound features

# READ side (model serving — the hot path)
f = r.hmget("features:user:123",
            ["txns_last_hour", "avg_txn_amount", "new_country"])
```

> **WHY THIS ENABLES REAL-TIME ML**
>
> When a request arrives ("is this transaction fraud?"), the model has only a few milliseconds. You can't run a heavy SQL aggregation in that window — but a Redis hash read returns in well under a millisecond. Pre-compute offline, read fast online.

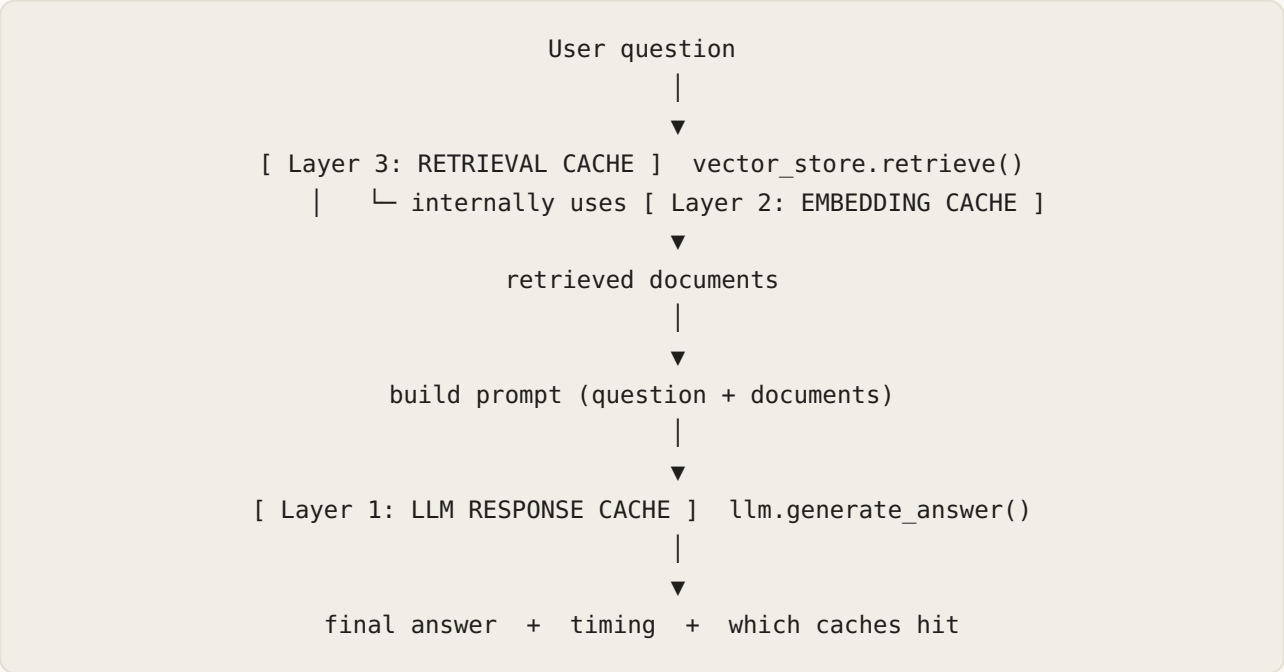
CHAPTER 6 · PROJECT 4

# The Cached RAG Chatbot

folder `project_rag_chatbot/`

This project combines the first three caches into one FastAPI application. A single question flows through all three layers, and the API reports exactly which layers were hits vs misses — so you can *see* caching at work.

## Architecture




## The files and what each one does

| File | Responsibility |
|---|---|
| `config.py` | All settings (Redis host, TTLs, models) read from environment. |
| `redis_client.py` | One shared Redis connection for the whole app. |
| `embeddings.py` | Text → vector, with the embedding cache (Layer 2). |
| `vector_store.py` | Tiny knowledge base + retrieval with the retrieval cache & versioned invalidation (Layer 3). |
| `llm.py` | Prompt building + answer generation with the LLM cache (Layer 1). |
| `rag_pipeline.py` | The conductor: runs all three layers in order, returns cache stats. |
| `main.py` | FastAPI endpoints: `/chat` , `/health` , `/admin/*` . |

**RUNS OFFLINE BY DEFAULT**

If you don't set `OPENAI_API_KEY`, the app uses deterministic *mock* models so you can run the entire pipeline with no key and no cost. Set a real key and the exact same code calls real OpenAI.

## The endpoints

| Endpoint | What it does |
|---|---|
| `GET /health` | Is the app up and is Redis reachable? |
| `POST /chat` | Ask a question; get a cached RAG answer + cache report. |
| `POST /admin/reindex` | Simulate a KB change → invalidates the retrieval cache. |
| `GET /admin/stats` | Count keys in each cache layer (uses safe `SCAN`). |

CHAPTER 7

# Running Everything

## Step 1 — Start Redis

```
# Easiest: Docker (from inside project_rag_chatbot/)
docker compose up -d

# Or, if Redis is installed locally:
redis-server --daemonize yes
redis-cli ping        # should print: PONG
```

## Step 2 — Try the standalone concepts

```
pip install redis
cd concepts
python 01_llm_cache.py          # watch MISS (slow) then HIT (instant)
python 02_embedding_cache.py    # single + batch caching
python 03_rag_cache.py          # retrieval cache + invalidation
python 04_feature_store_cache.py# write side / read side
```

## Step 3 — Run the RAG chatbot

```
cd project_rag_chatbot
pip install -r requirements.txt
uvicorn main:app --reload
# open http://localhost:8000/docs  (interactive API explorer)
```

## Step 4 — See the cache work

```
curl -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"question":"What is TTL?"}'
```

Call it **twice** with the same question and compare the `cache` field:

| Call | retrieval_cache_hit | llm_cache_hit | Speed |
|---|---|---|---|
| 1st | **false** | **false** | slower |
| 2nd | **true** | **true** | much faster |

Then call `POST /admin/reindex` and ask again — you'll see `retrieval_cache_hit` flip back to **false** (invalidated), while the LLM cache may still hit. That is versioned invalidation from Chapter 4, live.

CHAPTER 8

# Production Checklist & Takeaways

## Choosing a TTL per layer

| Layer | Suggested TTL | Reasoning |
|---|---|---|
| Embeddings | None (forever) | Deterministic; never stale. |
| LLM responses | Minutes–hours | Balance freshness vs. hit rate. |
| Retrieval | Minutes + versioning | Can go stale when KB changes. |
| Features | Match refresh cadence | Expire when new features are pushed. |

## Habits that separate seniors from beginners

- **Always namespace keys** ( `llm:` , `embedding:` , `rag:` ) so different data never collides.
- **Never use `KEYS` in production** — it blocks Redis. Use `SCAN` / `scan_iter` (as `/admin/stats` does).
- **Batch with `MGET` / `HMGET`** to cut round trips.
- **Always have an invalidation story** — TTL as a safety net, versioning for instant purges.
- **Measure your hit rate.** A cache you can't observe is a cache you can't trust; that's why every function here reports hit/miss.

### The five-second summary of Phase 5

LLM calls and embeddings are slow and expensive, but often repeated — so cache them with *cache-aside*. Embeddings never go stale (no TTL); LLM answers use TTL; retrieval needs TTL *and* versioned invalidation because the knowledge base changes; feature stores flip the pattern to pre-compute offline and read in milliseconds online. The RAG chatbot stacks the first three into one app so you can watch every layer hit and miss in real time.

## Where to go next → Phase 6

You've cached the *results* of AI work. Phase 6 (Agentic AI Redis) uses Redis for something new: **agent memory, conversation state, and checkpointing** — giving multi-agent systems a shared, durable brain. The caching instincts you built here carry straight over.